02157 Functional Programming
Week 6:
Lambda Calculus

Sebastian Mödersheim

October 9, 2026

Introduction

Today we look at the theoretical foundations of functional programming:

The Lambda Calculus

- Developed by Church, Turing, Rosser in the late 1930s
- Succinct and elegant way to think about computation
  - ★ Very small syntax
  - ★ Semantics is just one rule ... but there is a lot to say about it
- Basis for all functional languages
- All of F# can be just seen as syntactic sugar (abbreviating notation) for Lambda Calculus.

Since this all advanced and rooted in mathematical logic,

I will not ask about Lambda Calculus in the exam.

Please do not ignore the topic, it gives a deeper understanding!

Outline

1. Syntax
2. Semantics
3. Termination
4. Syntactic Sugar

Lambda Calculus: Syntax

Lambda Calculus

Given a set of variables (i.e., identifiers), the set of expressions is defined as follows.

- If x is a variable then $\boxed{x}$ is an expression.
- If e is an expression and x is a variable, then also $\boxed{\lambda x.e}$ is an expression.
- If e and e' are expressions, then also $\boxed{e\ e'}$ is an expression.
- If e is an expression, then also $\boxed{(e)}$ is an expression.

Written as a context-free grammar: $expr ::= var \mid \lambda var.expr \mid expr\ expr \mid (expr)$

Thus we only have:

- Variables
- Function defintion aka Lambda-Abstraction $\lambda x.e$
  - ★ corresponds to `fun x->e` in F#.
    (and everything else means literally the same as in F#.)
- Function Application
- Parentheses for structuring larger expressions

This is obviously the bare minumum for expressing functions

Lambda Calculus: Syntax

Lambda Calculus

Given a set of variables (i.e., identifiers), the set of expressions is defined as follows.

- If x is a variable then $\boxed{x}$ is an expression.
- If e is an expression and x is a variable, then also $\boxed{\lambda x.e}$ is an expression.
- If e and e' are expressions, then also $\boxed{e\ e'}$ is an expression.
- If e is an expression, then also $\boxed{(e)}$ is an expression.

Written as a context-free grammar: $expr ::= var \mid \lambda var.expr \mid expr\ expr \mid (expr)$

Thus we only have:

- Variables
- Function defintion aka Lambda-Abstraction $\lambda x.e$
  - ★ corresponds to `fun x->e` in F#.
    (and everything else means literally the same as in F#.)
- Function Application
- Parentheses for structuring larger expressions

This is obviously the bare minumum for expressing functions
... and we now show it is sufficient to express all computable functions.

What can we do?

$$expr ::= var \mid \lambda var.expr \mid expr\ expr \mid (expr)$$

This gives us almost nothing:

- There are no builtin datatypes like `int`
- There are no builtin functions like $+, \star$
- We cannot use recursion
  - ★ because you cannot recursively call a function if it is anonymous
- How can we really do anything in this Pure Lambda Calculus??

What can we do?

$$expr ::= var \mid \lambda var.expr \mid expr\ expr \mid (expr)$$

This gives us almost nothing:

- There are no builtin datatypes like `int`
- There are no builtin functions like $+, \star$
- We cannot use recursion
  - ★ because you cannot recursively call a function if it is anonymous
- How can we really do anything in this Pure Lambda Calculus??

Answer:

- Syntactic sugar: One can encode everything, and we see that soon
  - ★ theoretically appealing because we need only a very small language to describe a Turing-complete programming language
- Applied Lambda calculus: include primitive data types and operations
  - ★ more efficient than theoretical encodings
  - ★ for the sake of examples we will assume for now that we have integers and operators already

Outline

1. Syntax
2. **Semantics**
3. Termination
4. Syntactic Sugar

Evaluation of Lambda Expressions

Let us look at an example how we would expect Lambda expressions to evaluate:

Example

$$
\begin{aligned}
&(\lambda x.x * x)\ (2 + 5)\\
&\rightarrow (\lambda x.x * x)\ 7\\
&\rightarrow 7 * 7 \rightarrow 49
\end{aligned}
$$

Example

$$
\begin{aligned}
&(\lambda f.\lambda g.\lambda x.f\ (g\ x))\ (\lambda a.a + 2)\ (\lambda b.b * b)\ 3\\
&\rightarrow (\lambda g.\lambda x.(\lambda a.a + 2)\ (g\ x))\ (\lambda b.b * b)\ 3\\
&\rightarrow (\lambda x.(\lambda a.a + 2)\ ((\lambda b.b * b)\ x))\ 3\\
&\rightarrow (\lambda a.a + 2)\ ((\lambda b.b * b)\ 3)\\
&\rightarrow (\lambda a.a + 2)\ (3 * 3)\\
&\rightarrow (\lambda a.a + 2)\ 9\\
&\rightarrow 9 + 2 \rightarrow 11
\end{aligned}
$$

How can we make this precise? That's the semantics.

Semantics: β-Reduction

We have just used a relation $\rightarrow$ to mean evaluating expressions. Formal definition

Definition (β-Reduction)

A term of the form $(\lambda x.e)\ e'$ is called a β-redex.
A β-reduction is of the form

$$(\lambda x.e)\ e' \rightarrow e[x \mapsto e']$$

where $e[x \mapsto e']$ roughly means: take e and replace every occurrence of x by e'.

Example: $(\lambda x.\lambda y.x + y)\ 7\ 3$
$\rightarrow (\lambda y.x + y)[x \mapsto 7]\ 3 = (\lambda y.7 + y)\ 3$
$\rightarrow (7 + y)[y \mapsto 3] = 7 + 3 = 10$

Semantics: β-Reduction

We have just used a relation $\rightarrow$ to mean evaluating expressions. Formal definition

Definition (β-Reduction)

A term of the form $(\lambda x.e)\ e'$ is called a β-redex.
A β-reduction is of the form

$$(\lambda x.e)\ e' \rightarrow e[x \mapsto e']$$

where $e[x \mapsto e']$ roughly means: take e and replace every occurrence of x by e'.

Example: $(\lambda x.\lambda y.x + y)\ 7\ 3$
$\rightarrow (\lambda y.x + y)[x \mapsto 7]\ 3 = (\lambda y.7 + y)\ 3$
$\rightarrow (7 + y)[y \mapsto 3] = 7 + 3 = 10$

Why roughly?

Semantics: β-Reduction

We have just used a relation $\rightarrow$ to mean evaluating expressions. Formal definition

Definition (β-Reduction)

A term of the form $(\lambda x.e)\ e'$ is called a β-redex.
A β-reduction is of the form

$$(\lambda x.e)\ e' \rightarrow e[x \mapsto e']$$

where $e[x \mapsto e']$ roughly means: take e and replace every occurrence of x by e'.

Example: $(\lambda x.\lambda y.x + y)\ 7\ 3$
$\rightarrow (\lambda y.x + y)[x \mapsto 7]\ 3 = (\lambda y.7 + y)\ 3$
$\rightarrow (7 + y)[y \mapsto 3] = 7 + 3 = 10$

Why roughly?

- We must be careful not to capture free variables e.g.,
  $(\lambda x.\lambda y.x + y)\ y \not\rightarrow \lambda y.y + y$

Semantics: β-Reduction

We have just used a relation $\rightarrow$ to mean evaluating expressions. Formal definition

Definition (β-Reduction)

A term of the form $(\lambda x.e)\ e'$ is called a β-redex.
A β-reduction is of the form

$$(\lambda x.e)\ e' \rightarrow e[x \mapsto e']$$

where $e[x \mapsto e']$ roughly means: take e and replace every occurrence of x by e'.

Example: $(\lambda x.\lambda y.x + y)\ 7\ 3$
$\rightarrow (\lambda y.x + y)[x \mapsto 7]\ 3 = (\lambda y.7 + y)\ 3$
$\rightarrow (7 + y)[y \mapsto 3] = 7 + 3 = 10$

Why roughly?

- We must be careful not to capture free variables e.g.,
  $(\lambda x.\lambda y.x + y)\ y \not\rightarrow \lambda y.y + y$
- In such a case an α-renaming is necessary, renaming the bound variable y to something else:
  $(\lambda x.\lambda y.x + y)\ y \equiv (\lambda x.\lambda z.x + z)\ y \rightarrow \lambda z.y + z$

Semantics: Normal Form

Definition (Normal-Form)

An expression e is in normal form, if no subterm of e is a β-redex.

Example: $(\lambda x.\lambda y.x + y)\ 7\ 3$
$\rightarrow (\lambda y.x + y)[x \mapsto 7]\ 3 = (\lambda y.7 + y)\ 3$
$\rightarrow (7 + y)[y \mapsto 3] = 7 + 3 = 10$ *is in normal form*

Theorem (Church-Rosser)

β-reduction is confluent (modulo α renaming $\equiv$), i.e.,

- *if $e \rightarrow^* e_1$ and $e \rightarrow^* e_2$*
- *and e_1 and e_2 are in normal form,*
- *then $e_1 \equiv e_2$*

Example (assuming we already have arithmetic):
consider all the reductions of $(\lambda x.x \star x)(3 + 5)$:

$$(\lambda x.x \star x)\ (3 + 5) \rightarrow (3 + 5) \star (3 + 5) \rightarrow 8 \star (3 + 5) \rightarrow 8 \star 8 \rightarrow 64$$
$$(\lambda x.x \star x)\ (3 + 5) \rightarrow (3 + 5) \star (3 + 5) \rightarrow (3 + 5) \star 8 \rightarrow 8 \star 8 \rightarrow 64$$
$$(\lambda x.x \star x)\ (3 + 5) \rightarrow (\lambda x.x \star x)\ 8 \rightarrow 8 \star 8 \rightarrow 64$$

Semantics: Evaluation Strategy

- Evaluation strategy: which reduction is performed first.
- Confluence means that the result does not depend on the evaluation strategy.
- However, whether evaluation terminates can depend on the evaluation strategy.
- This is a complicated issue that we deal with in another lecture.
- For now we take the strategy outermost-leftmost:
  - ★ The redex $(\lambda x.e_1)\ e_2 \to e_1[x \mapsto e_2]$ has priority over all redices in e_1 and e_2
  - ★ In $e_1\ e_2$ we first normalize e_1 and then e_2.
- This strategy is optimal for termination
  - ★ if there is a normal form, we will reach it with this strategy.

Outline

1. Syntax
2. Semantics
3. **Termination**
4. Syntactic Sugar

Semantics: Termination

Not every term has a normal form:

$$\textit{let } e_\perp = (\lambda x.x\ x)\ (\lambda x.x\ x)$$

Semantics: Termination

Not every term has a normal form:

$$\textit{let } e_\perp = (\lambda x.x\ x)\ (\lambda x.x\ x)$$

- This is a "strange" term: x is both used as a function and as an argument to that function!
- This would not be allowed by the type system in F#
  - ★ because $x : \alpha \to \beta$ and $x : \alpha$ is not possible at the same time.

Semantics: Termination

Not every term has a normal form:

$$let\ e_\perp = (\lambda x.x\ x)\ (\lambda x.x\ x)$$

- This is a "strange" term: x is both used as a function and as an argument to that function!
- This would not be allowed by the type system in F#
  - ★ because $x : \alpha \to \beta$ and $x : \alpha$ is not possible at the same time.
- However, Pure Lambda Calculus is untyped; writing such strange terms is perfectly legal.

Semantics: Termination

Not every term has a normal form:

$$\textit{let } e_\perp = (\lambda x.x\ x)\ (\lambda x.x\ x)$$

- This is a "strange" term: x is both used as a function and as an argument to that function!
- This would not be allowed by the type system in F#
  - ★ because $x : \alpha \to \beta$ and $x : \alpha$ is not possible at the same time.
- However, Pure Lambda Calculus is untyped; writing such strange terms is perfectly legal.
- It holds that:

$$e_\perp = (\lambda x.x\ x)\ (\lambda x.x\ x) \to (x\ x)[x \mapsto (\lambda x.x\ x)] = (\lambda x.x\ x)\ (\lambda x.x\ x) = e_\perp$$

Semantics: Termination

Not every term has a normal form:

$$\textit{let } e_\perp = (\lambda x.x\ x)\ (\lambda x.x\ x)$$

- This is a "strange" term: x is both used as a function and as an argument to that function!
- This would not be allowed by the type system in F#
  - ★ because $x : \alpha \to \beta$ and $x : \alpha$ is not possible at the same time.
- However, Pure Lambda Calculus is untyped; writing such strange terms is perfectly legal.
- It holds that:

$$e_\perp = (\lambda x.x\ x)\ (\lambda x.x\ x) \to (x\ x)[x \mapsto (\lambda x.x\ x)] = (\lambda x.x\ x)\ (\lambda x.x\ x) = e_\perp$$

- Thus, $e_\perp \to e_\perp$ and we never reach a normal form.

Semantics: Termination

Not every term has a normal form:

$$\textit{let } e_\perp = (\lambda x.x\ x)\ (\lambda x.x\ x)$$

- This is a "strange" term: x is both used as a function and as an argument to that function!
- This would not be allowed by the type system in F#
  - ★ because $x : \alpha \to \beta$ and $x : \alpha$ is not possible at the same time.
- However, Pure Lambda Calculus is untyped; writing such strange terms is perfectly legal.
- It holds that:

$$e_\perp = (\lambda x.x\ x)\ (\lambda x.x\ x) \to (x\ x)[x \mapsto (\lambda x.x\ x)] = (\lambda x.x\ x)\ (\lambda x.x\ x) = e_\perp$$

- Thus, $e_\perp \to e_\perp$ and we never reach a normal form.
- Claim: in every reasonable programming language it must be possible to specify a program that loops forever.
  Of course, $e_\perp$ is not really useful – similar to `while(true){}` in C/Java.

Semantics: Termination

Not every term has a normal form:

$$\textit{let } e_\perp = (\lambda x.x\ x)\ (\lambda x.x\ x)$$

- This is a "strange" term: x is both used as a function and as an argument to that function!
- This would not be allowed by the type system in F#
  - ★ because $x : \alpha \to \beta$ and $x : \alpha$ is not possible at the same time.
- However, Pure Lambda Calculus is untyped; writing such strange terms is perfectly legal.
- It holds that:

$$e_\perp = (\lambda x.x\ x)\ (\lambda x.x\ x) \to (x\ x)[x \mapsto (\lambda x.x\ x)] = (\lambda x.x\ x)\ (\lambda x.x\ x) = e_\perp$$

- Thus, $e_\perp \to e_\perp$ and we never reach a normal form.
- Claim: in every reasonable programming language it must be possible to specify a program that loops forever.
  Of course, $e_\perp$ is not really useful – similar to `while(true){}` in C/Java.
- We see later that we can use something similar to $e_\perp$ for recursion!

Semantics: The Semantics Function

We can now define the semantics, or meaning, of a lambda expression as its normal form, if it exists. Otherwise, its semantics is $\bot$ (undefined).

Definition

For a lambda expression e its semantics $[\![e]\!]$ is either

- e' if $e \to^\star e'$ and e' in normal form,
- $\bot$ if there is no such e'.

Thus $[\![\cdot]\!]$ is a partial function from expressions to expressions.

Note that if normal form e' exists, then it is unique (modulo α-renaming).
Examples:

- $[\![(\lambda x.\lambda y.x)\ \mathit{true}\ \mathit{false}]\!] = \mathit{true}$
- $[\![(\lambda x.\lambda y.x)\ e_\bot\ \mathit{false}]\!] = \bot$ (undefined, does not terminate)
- $[\![(\lambda x.\lambda y.x)\ \mathit{true}\ e_\bot]\!] = \mathit{true}$

The Halting Problem

Reformulated for Lambda Calculus

Definition (The Halting Problem)

$$Halt(e) = \begin{cases} true & \text{if } [\![e]\!] \neq \bot \\ false & \text{otherwise} \end{cases}$$

The Halting Problem

Reformulated for Lambda Calculus

Definition (The Halting Problem)

$$Halt(e) = \begin{cases} true & \text{if } [\![e]\!] \neq \bot \\ false & \text{otherwise} \end{cases}$$

Theorem (Turing)

Halt is undecidable, i.e., there is no expression H such that $[\![H\ e]\!] = Halt(e)$ for every expression e.

The Halting Problem

Reformulated for Lambda Calculus

Definition (The Halting Problem)

$$Halt(e) = \begin{cases} true & \text{if } [\![e]\!] \neq \bot \\ false & \text{otherwise} \end{cases}$$

Theorem (Turing)

Halt is undecidable, i.e., there is no expression H such that $[\![H\ e]\!] = Halt(e)$ for every expression e.

Proof.

Suppose there is an expression H such that for every expression e, $[\![H\ e]\!] = Halt(e)$.

The Halting Problem

Reformulated for Lambda Calculus

Definition (The Halting Problem)

$$Halt(e) = \begin{cases} true & \text{if } [\![e]\!] \neq \bot \\ false & \text{otherwise} \end{cases}$$

Theorem (Turing)

Halt is undecidable, i.e., there is no expression H such that $[\![H\ e]\!] = Halt(e)$ for every expression e.

Proof.

Suppose there is an expression H such that for every expression e, $[\![H\ e]\!] = Halt(e)$. Consider the expression $Test = \lambda e.if\ H\ (e\ e)\ then\ e_{\bot}\ else\ true$

The Halting Problem

Reformulated for Lambda Calculus

Definition (The Halting Problem)

$$Halt(e) = \begin{cases} true & \text{if } [\![e]\!] \neq \bot \\ false & \text{otherwise} \end{cases}$$

Theorem (Turing)

Halt is undecidable, i.e., there is no expression H such that $[\![H\ e]\!] = Halt(e)$ *for every expression e.*

Proof.

Suppose there is an expression H such that for every expression e, $[\![H\ e]\!] = Halt(e)$.
Consider the expression $Test = \lambda e.if\ H\ (e\ e)\ then\ e_\bot\ else\ true$
Then the following statements are equivalent:

- $Halt(Test\ \ Test) = true$
- $[\![Test\ \ Test]\!] \neq \bot$
- $[\![if\ H\ (Test\ \ Test)\ then\ e_\bot\ else\ true]\!] \neq \bot$
- $[\![H\ (Test\ \ Test)]\!] = false$
- $Halt(Test\ \ Test) = false$

The Halting Problem

Reformulated for Lambda Calculus

Definition (The Halting Problem)

$$Halt(e) = \begin{cases} true & \text{if } [\![e]\!] \neq \bot \\ false & \text{otherwise} \end{cases}$$

Theorem (Turing)

Halt is undecidable, i.e., there is no expression H such that $[\![H\ e]\!] = Halt(e)$ *for every expression e.*

Proof.

Suppose there is an expression H such that for every expression e, $[\![H\ e]\!] = Halt(e)$.
Consider the expression $Test = \lambda e.if\ H\ (e\ e)\ then\ e_\bot\ else\ true$
Then the following statements are equivalent:

- $Halt(Test\ \ Test) = true$
- $[\![Test\ \ Test]\!] \neq \bot$
- $[\![if\ H\ (Test\ \ Test)\ then\ e_\bot\ else\ true]\!] \neq \bot$
- $[\![H\ (Test\ \ Test)]\!] = false$
- $Halt(Test\ \ Test) = false$ q.e.a.

Outline

1. Syntax
2. Semantics
3. Termination
4. **Syntactic Sugar**

Pure Lambda Calculus

$$expr ::= var \mid \lambda var.expr \mid expr\ expr \mid (expr)$$

We have before assumed that we have at least integers and arithmetic operations.
We want to now show that we can encode in Pure Lambda Calculus:

- Algebraic Data Types
  - ★ and thus also ints, strings, lists, ...
- Recursion
  - ★ and thus also addition, multiplication, list operations, ...

Notation:

- Similar to F#, let us write $let\ f = e$ to (non-recursively) define a function f by the expression e.
- This means nothing but an abbreviation: we can use in the following the identifier f instead of writing out expression e.
- Similar to F#, let us write $let\ f\ x_1\ x_2\ \ldots\ x_n = e$ for $let\ f = \lambda x_1.\lambda x_2.\ \cdots\ \lambda x_n.e$

Example: Quantified Boolean Logic

Here is an idea how to encode a data type like boolean by continuation-style

Encoding of Quantified Boolean Logic

$$
\begin{array}{lcl}
\mathit{let}\ \mathit{true} & = & \lambda x.\lambda y.x \\
\mathit{let}\ \mathit{false} & = & \lambda x.\lambda y.y \\
\mathit{let}\ \mathit{not}\ b & = & \\
\mathit{let}\ \mathit{and}\ b_1\ b_2 & = & \\
\mathit{let}\ \mathit{or}\ b_1\ b_2 & = & \\
\mathit{let}\ \mathit{exists}\ p & = & \\
\mathit{let}\ \mathit{forall}\ p & = &
\end{array}
$$

Note: Consider the words if then else as pure commentary—they just makes it more intuitive to read.

Example: Quantified Boolean Logic

Here is an idea how to encode a data type like boolean by continuation-style

Encoding of Quantified Boolean Logic

$$
\begin{array}{lcl}
\mathit{let}\ \mathit{true} & = & \lambda x.\lambda y.x \\
\mathit{let}\ \mathit{false} & = & \lambda x.\lambda y.y \\
\mathit{let}\ \mathit{not}\ b & = & \text{if } b \text{ then } \mathit{false} \text{ else } \mathit{true} \\
\mathit{let}\ \mathit{and}\ b_1\ b_2 & = & \\
\mathit{let}\ \mathit{or}\ b_1\ b_2 & = & \\
\mathit{let}\ \mathit{exists}\ p & = & \\
\mathit{let}\ \mathit{forall}\ p & = &
\end{array}
$$

Note: Consider the words if then else as pure commentary—they just makes it more intuitive to read.

Example: Quantified Boolean Logic

Here is an idea how to encode a data type like boolean by continuation-style

Encoding of Quantified Boolean Logic

$$
\begin{array}{lcl}
\mathit{let}\ \mathit{true} & = & \lambda x.\lambda y.x \\
\mathit{let}\ \mathit{false} & = & \lambda x.\lambda y.y \\
\mathit{let}\ \mathit{not}\ b & = & \text{if } b \text{ then } \mathit{false} \text{ else } \mathit{true} \\
\mathit{let}\ \mathit{and}\ b_1\ b_2 & = & \text{if } b_1 \text{ then } b_2 \text{ else } \mathit{false} \\
\mathit{let}\ \mathit{or}\ b_1\ b_2 & = & \\
\mathit{let}\ \mathit{exists}\ p & = & \\
\mathit{let}\ \mathit{forall}\ p & = &
\end{array}
$$

Note: Consider the words if then else as pure commentary—they just makes it more intuitive to read.

Example: Quantified Boolean Logic

Here is an idea how to encode a data type like boolean by continuation-style

Encoding of Quantified Boolean Logic

$$
\begin{array}{lcl}
\mathit{let}\ \mathit{true} & = & \lambda x.\lambda y.x \\
\mathit{let}\ \mathit{false} & = & \lambda x.\lambda y.y \\
\mathit{let}\ \mathit{not}\ b & = & \text{if } b \text{ then } \mathit{false} \text{ else } \mathit{true} \\
\mathit{let}\ \mathit{and}\ b_1\ b_2 & = & \text{if } b_1 \text{ then } b_2 \text{ else } \mathit{false} \\
\mathit{let}\ \mathit{or}\ b_1\ b_2 & = & \text{if } b_1 \text{ then } \mathit{true} \text{ else } b_2 \\
\mathit{let}\ \mathit{exists}\ p & = & \\
\mathit{let}\ \mathit{forall}\ p & = &
\end{array}
$$

Note: Consider the words if then else as pure commentary—they just makes it more intuitive to read.

Example: Quantified Boolean Logic

Here is an idea how to encode a data type like boolean by continuation-style

Encoding of Quantified Boolean Logic

$$
\begin{array}{lcl}
\mathit{let}\ \mathit{true} & = & \lambda x.\lambda y.x \\
\mathit{let}\ \mathit{false} & = & \lambda x.\lambda y.y \\
\mathit{let}\ \mathit{not}\ b & = & \text{if } b \text{ then } \mathit{false} \text{ else } \mathit{true} \\
\mathit{let}\ \mathit{and}\ b_1\ b_2 & = & \text{if } b_1 \text{ then } b_2 \text{ else } \mathit{false} \\
\mathit{let}\ \mathit{or}\ b_1\ b_2 & = & \text{if } b_1 \text{ then } \mathit{true} \text{ else } b_2 \\
\mathit{let}\ \mathit{exists}\ p & = & \mathit{or}\ (p\ \mathit{true})\ (p\ \mathit{false}) \\
\mathit{let}\ \mathit{forall}\ p & = &
\end{array}
$$

Note: Consider the words if then else as pure commentary—they just makes it more intuitive to read.

Example: Quantified Boolean Logic

Here is an idea how to encode a data type like boolean by continuation-style

Encoding of Quantified Boolean Logic

$$
\begin{array}{lcl}
\mathit{let}\ \mathit{true} & = & \lambda x.\lambda y.x \\
\mathit{let}\ \mathit{false} & = & \lambda x.\lambda y.y \\
\mathit{let}\ \mathit{not}\ b & = & \text{if } b \text{ then } \mathit{false} \text{ else } \mathit{true} \\
\mathit{let}\ \mathit{and}\ b_1\ b_2 & = & \text{if } b_1 \text{ then } b_2 \text{ else } \mathit{false} \\
\mathit{let}\ \mathit{or}\ b_1\ b_2 & = & \text{if } b_1 \text{ then } \mathit{true} \text{ else } b_2 \\
\mathit{let}\ \mathit{exists}\ p & = & \mathit{or}\ (p\ \mathit{true})\ (p\ \mathit{false}) \\
\mathit{let}\ \mathit{forall}\ p & = & \mathit{and}\ (p\ \mathit{true})\ (p\ \mathit{false})
\end{array}
$$

Note: Consider the words if then else as pure commentary—they just makes it more intuitive to read.

Example: Quantified Boolean Logic

Here is an idea how to encode a data type like boolean by continuation-style

Encoding of Quantified Boolean Logic

$$
\begin{array}{lcl}
\mathit{let}\ \mathit{true} & = & \lambda x.\lambda y.x \\
\mathit{let}\ \mathit{false} & = & \lambda x.\lambda y.y \\
\mathit{let}\ \mathit{not}\ b & = & \text{if } b \text{ then } \mathit{false} \text{ else } \mathit{true} \\
\mathit{let}\ \mathit{and}\ b_1\ b_2 & = & \text{if } b_1 \text{ then } b_2 \text{ else } \mathit{false} \\
\mathit{let}\ \mathit{or}\ b_1\ b_2 & = & \text{if } b_1 \text{ then } \mathit{true} \text{ else } b_2 \\
\mathit{let}\ \mathit{exists}\ p & = & \mathit{or}\ (p\ \mathit{true})\ (p\ \mathit{false}) \\
\mathit{let}\ \mathit{forall}\ p & = & \mathit{and}\ (p\ \mathit{true})\ (p\ \mathit{false})
\end{array}
$$

Note: Consider the words if then else as pure commentary—they just makes it more intuitive to read.

Example: $\mathit{forall}\ (\lambda x.\mathit{or}\ x\ (\mathit{not}\ x)) = ?$

Encoding Arithmetic

Algebraic Data Type	Encoding in λ-Calculus
$type\ Bool = true$	$let\ true = \lambda x.\lambda y.x$
$\quad \mid false$	$let\ false = \lambda x.\lambda y.y$
$type\ Nat = 0$	$let\ 0 = \lambda x.\lambda y.x$
$\quad \mid s\ of\ Nat$	$let\ s\ n = \lambda x.\lambda y.y\ n$

- Idea: Booleans we had previously encoded as functions that take two continuations x and y: x for the case true and y for the case false, and we call the respective continuation.
- For natural numbers, we can also take two continuations x and y, x for the case 0 and y for the non-zero case:
  - ★ $s\ n$ represents "successor of n" i.e. $n + 1$.
  - ★ E.g. 3 is thus encoded as $s(s(s(0)))$. This is unary encoding, terrible for efficiency, but we get back to that.
- For number $s\ n$, the continuation y gets n as an argument. Why?
  - ★ This allows in computations to further work with the predecessor n, e.g. in a recursion.

Encoding Arithmetic

To work more easily with the continuations we can define Pattern matching as syntactic sugar:

$$\boxed{\begin{array}{l} match\ e\ with \\ |\ 0 \quad \rightarrow e_0 \\ |\ s\ n \rightarrow e_s \end{array}}\ \text{as syntactic sugar for}\ \boxed{e\ e_0\ (\lambda n.e_s)}$$

Now we can define addition (and similarly multiplication and exponentiation):

$$\begin{array}{l} let\ rec\ add\ n\ m\ =\ match\ n\ with \\ \quad |\ 0 \quad \rightarrow m \\ \quad |\ s\ n' \rightarrow s\ (add\ n'\ m) \end{array}$$

Encoding Arithmetic

To work more easily with the continuations we can define Pattern matching as syntactic sugar:

$$\boxed{\begin{array}{l} match\ e\ with \\ |\ 0 \quad \rightarrow e_0 \\ |\ s\ n \rightarrow e_s \end{array}} \text{ as syntactic sugar for } \boxed{e\ e_0\ (\lambda n.e_s)}$$

Now we can define addition (and similarly multiplication and exponentiation):

$$\begin{array}{l} let\ rec\ add\ n\ m\ =\ match\ n\ with \\ \quad |\ 0 \quad \rightarrow m \\ \quad |\ s\ n' \rightarrow s\ (add\ n'\ m) \end{array}$$

. . . but this requires that we can somehow express recursion in Lambda Calculus.

Recursion in Lambda Calculus

Remember $e_\perp = (\lambda x.x\ x)\ (\lambda x.x\ x)$ that creates a useless infinite loop.

But a similar construction can actually do something useful:

Definition (Fixedpoint Combinator)

$$let\ Y = \lambda F.\ (\lambda x.F\ (x\ x))\ (\lambda x.F\ (x\ x))$$

Recursion in Lambda Calculus

$$Y\ F \to F(Y\ F)$$

With that we can express recursion:

Definition

$$\begin{array}{l} \mathit{let}\ \mathit{add0}\ f\ n\ m\ =\ \mathit{match}\ n\ \mathit{with} \\ \quad \mid 0 \quad \to m \\ \quad \mid s\ n' \to s\ (f\ n'\ m) \\ \mathit{let}\ \mathit{add} = Y\ \mathit{add0} \end{array}$$

Observe that there is no recursion in this definition.

Similar Definitions

Arithmetic

One can in similar recursive definitions define

- Multiplication, exponentiation, comparison, subtraction and division as far as defined on natural numbers.
- Because of the unary representation this is extremely inefficient, but we can encode natural numbers arithmetic.
- With that and recursion we are Turing complete.
- One can also make a asymptotically optimal implementation with a binary representation — i.e., a lists of bools.

Encoding Lists

We use the same trick to encode algebraic data types:

Encoding Lists

Algebraic Data Type	Encoding in λ-Calculus
$type\ \alpha\ List = Empty$	$let\ Empty = \lambda x.\lambda y.x$
$\quad \mid Cons\ of\ (\alpha, \alpha\ List)$	$let\ Cons\ a\ as = \lambda x.\lambda y.y\ a\ as$

$$\boxed{\begin{array}{l} match\ e\ with \\ \mid Empty \rightarrow e_0 \\ \mid Cons\ a\ as \rightarrow e_s \end{array}}\ \text{as syntactic sugar for}\ \boxed{e\ e_0\ (\lambda a.\lambda as.e_s)}$$

Note that this is untyped Lambda Calculus; the lists work for any type (including inhomogeneous).

Summing a list of natural numbers

```
let rec suml l = match l with
  | Empty → 0
  | Cons a as → add a (suml as)
```

Map

```
let rec map f l = match l with
  | Empty → Empty
  | Cons a as → Cons (f a) (map f as)
```

Exercises

Make some definitions in Pure Lambda Calculus - you can use all definitions we have introduced in the lecture

- Define the boolean function *xor* (exclusive or). One can define *xor a b* as "*a* or *b*, but not both *a* and *b*". Try to define it in a way that it does not need so many reductions.
- A definition for addition was given using the fixedpoint combinator Y (instead of recursion). Give a similar definition for multiplication and factorial, using the fixedpoint combinator Y. (Multiplication can use addition of course, and factorial can use multiplication.)
- For lists, define *fold* and use it to re-define *suml*.

From the summer 2025 exam
https://www.imm.dtu.dk/ mire/FSharpBook/OldExams/02157-2025summer.pdf
consider

- Problem 2 without subtask 4 (we come to tail recursion later)
- Problem 3 without subtask 2 (same reason)